GraphIAM: Two-Stage Algorithm for Improving Class-Imbalanced Node Classification on Attribute-Missing Graphs

Riting Xia
Inner Mongolia University
College of Computer Science
Hohhot, Inner Mongolia Autonomous Region, China
xiart@imu.edu.cn

Chunxu Zhang
Jilin University
School of Computer Science and Technology
Changchun, Jilin, China
zhangchunxu@jlu.edu.cn

Xueyan Liu
Jilin University
School of Computer Science and Technology
Changchun, Jilin, China
xueyanliu@jlu.edu.cn

Anchen Li
Jilin University
School of Computer Science and Technology
Changchun, Jilin, China
liac@jlu.edu.cn

Yan Zhang*
Inner Mongolia University
College of Computer Science
Hohhot, Inner Mongolia Autonomous Region, China
zhangyan@imu.edu.cn

Abstract

Addressing class-imbalanced graphs is a challenging task due to the involvement of both node attributes and graph structures. Existing works on class-imbalanced graphs simply assume that all node attributes are available. However, in real-world graphs, many nodes may lack attributes due to privacy issues or missing data, making class-imbalanced graph learning more challenging. In this paper, we propose GraphIAM, a novel two-stage algorithm for improving class-imbalanced node classification on attribute-missing graphs. In the pre-training phase, GraphIAM adopts graph contrastive learning with oversampling to tackle both attribute-missing and class-imbalanced issues. During fine-tuning, an adapter mechanism is introduced to learn node representations, alleviating the generalization gap between pre-training and downstream tasks. Experimental results on benchmark datasets demonstrate that our method achieves state-of-the-art performance, outperforming class-imbalanced graph learning approaches by 5% in F Score on graphs with severe attribute missingness.

CCS Concepts

• **Computing methodologies** → **Machine learning**.

Keywords

Class-imbalanced graphs; attribute-missing graphs; two-stage algorithm; oversampling; node classification

*Corresponding author

CIKM '25, Seoul, Republic of Korea

ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3761086

ACM Reference Format:
Riting Xia, Chunxu Zhang, Xueyan Liu, Anchen Li, and Yan Zhang. 2025. GraphIAM: Two-Stage Algorithm for Improving Class-Imbalanced Node Classification on Attribute-Missing Graphs. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3746252.3761086

1 Introduction

Class-imbalanced graphs [19, 22], where the numbers of nodes in certain classes may be significantly fewer compared to others, are prevalent in real-world applications. For example, in citation networks, popular research topics tend to have more publications than lesser-known topics. Similarly, in social networks, most users are authentic, with only a small proportion representing fake accounts. As the class-imbalanced graph analysis continues to grow across different domains [7, 16, 36, 41], the task of class-imbalanced node classification on graphs has garnered considerable interest.

Several effective graph machine learning methods have been proposed for class-imbalanced node classification on graphs. These existing approaches are generally categorized into data-level and algorithm-level strategies [20]. Algorithm-level methods typically capitalize on the inherent characteristics of class-imbalanced graphs to improve the performance of existing algorithms [32]. Data-level methods focus on modifying the training data in feature or label space to achieve class balance, often through data generation and pseudo-label generation techniques [26]. Existing class-imbalanced methods can achieve promising performance in node classification tasks when graphs have complete node attributes.

However, in practical graphs, some nodes may have missing attributes due to privacy protection or data loss, which constitute attribute-missing graphs [5]. For example, in citation networks, many papers may be inaccessible due to copyright protection. This results in the information of minority nodes becoming even more scarce, posing greater challenges to existing class-imbalanced graph learning methods that do not account for attribute missingness (as shown in Figure 1). To our knowledge, there has been no prior research on addressing class-imbalanced graphs in which attributes

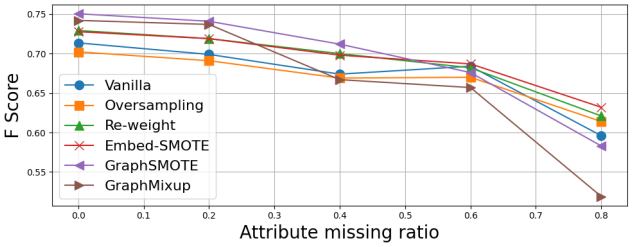


Figure 1: F Score of existing class-imbalanced methods. Missing attributes degrade existing methods, with performance declining as missing ratio increases.

of some nodes are completely missing. Another related topic is attribute completion for graphs [5, 29], which focuses on filling in missing attributes but often overlooks the class-imbalanced issue. In this work, ***we aim to mitigate the class imbalance on graphs while tackling the challenge of missing attributes.***

Data generation methods [20], which aim to balance the class distribution in graphs by augmenting the minority nodes, have proven to be effective in addressing class-imbalanced node classification problems. It is worth noting that many data generation methods adopt a two-stage model, where they initially train the model to learn node representations in the pre-training stage and subsequently classify the node representations through fine-tuning. For example, the data generation methods of GraphSMOTE [42] and GraphMixup [33] follow the two-stage process. The advantage of the two-stage approach lies in its efficacy for class-imbalanced node classification tasks within graphs, an efficacy that has also been demonstrated in other domains, such as image classification [6, 13, 18]. Additionally, to mitigate the impact of missing attributes on node classification results, it is crucial to complete the nodes' attributes during the node representation learning, in line with the pre-training stage of the two-stage approach. However, existing two-stage methods on graphs fail to consider the issue of missing attributes and are not directly applicable to addressing the problem we posed.

Based on the above analysis, we introduce a novel problem of class-imbalanced node classification on attribute-missing graphs. We propose a two-stage data generation model (GraphIAM) to tackle this problem. Our goal is to overcome the two main challenges of existing two-stage models: (1) during the pre-training stage, addressing ***how to acquire robust and class-balanced node representations for graphs with missing attributes***, and (2) in the fine-tuning stage, addressing ***how to enhance the generalization capability of node representations for downstream tasks***. To tackle the first challenge, we utilize graph contrastive learning to obtain robust and class-balanced node representations. Specifically, we first create structural and attribute augmentation graphs through two data augmentation strategies. Subsequently, we encode the node representations from these two types of graphs and integrate the subgraph of attribute-missing nodes from the structural augmentation graph into the attribute augmentation graph. We then employ oversampling to obtain node representations with relatively balanced class distributions for both structural and attribute augmentation graphs. In addition, we train node representations using contrastive loss and structure reconstruction loss. For the second challenge, we propose adopting the adapter to enhance the generalization ability of the model. Specifically, we incorporate task-specific parameters into the pre-trained stage and freeze the main model during fine-tuning, solely training the task-specific ones to enhance generalization ability.

The main contributions of this article are threefold:

- We study a novel graph learning problem where the goal is to perform class-imbalanced node classification on attribute-missing graphs. This setting extends previous research by jointly considering class imbalance and attribute absence, which are common in real-world graphs.
- We propose a novel two-stage model (GraphIAM), which decouples pre-training and fine-tuning stages to achieve improved node classification results. To obtain enhanced node representations, we design a graph contrastive learning method for node representation learning. To enhance the model's generalization ability, we introduce an adapter. This method addresses the issue of class imbalance while completing missing attributes. Our method is the first to utilize adapters to address class-imbalanced graphs.
- Extensive benchmarks show GraphIAM's effectiveness in class-imbalanced node classification for attribute-missing graphs, particularly exhibiting more pronounced performance under severe class imbalance and attribute absence.

2 Related Work

2.1 Class-Imbalanced Graph Learning

With the increasing focus on class-imbalanced node classification in graphs, researchers have proposed numerous approaches, which can be categorized into two paradigms: algorithmic-level and data-level algorithms. Algorithmic-level methods aim to tackle class-imbalanced issues by modifying algorithms [4, 10, 25]. Data-level methods typically leverage generative techniques to balance the number of nodes or labels across different classes. This category of methods is primarily divided into pseudo-label and data generation methods. Pseudo-label generation methods involve predicting the labels of unlabeled nodes using labeled nodes, which are then utilized for model training [43]. Data generation methods aim to balance the class distribution by increasing the number of minority nodes [17, 22, 23, 31, 33, 36, 42].

In this paper, we focus on utilizing data generation methods to tackle class-imbalanced node classification on attribute-missing graphs. We observe that many existing methods rely on two-stage training, effective but neglecting missing attributes. They also struggle with generalization due to pre-training and task inconsistency, impacting node classification.

2.2 Attribute Completion on Graphs

Attribute-missing graphs, where some nodes lack attributes, are commonly encountered in the real-world [9, 11, 28, 34, 35]. Several graph machine learning models have been proposed to address this challenge. Chen et al. [5] highlighted the prevalence of attribute-missing graphs and introduced the Structure-attribute Transformer

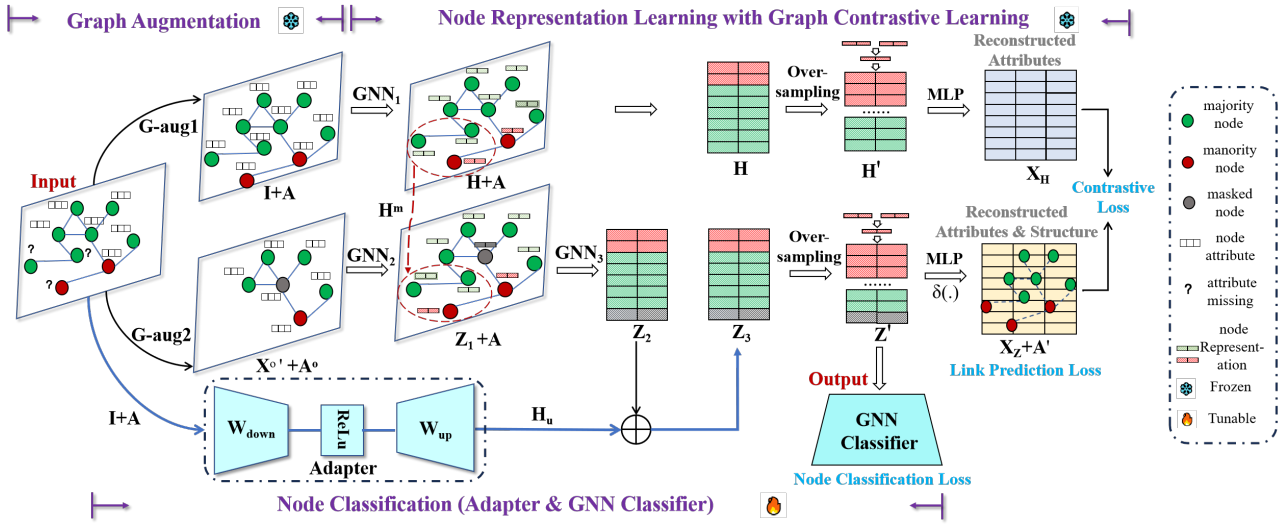


Figure 2: Architecture of GraphIAM.

(SAT), which completes missing attributes by assuming a shared latent space for structure and attributes. Jin et al. [12] introduced the Amer framework, a game-based approach that combines attribute completion and network representation learning through mutual information maximization. Tu et al. [29] introduced the Initializing Then Refining (ITR) mechanism, which uses reliable attribute and structural information to learn the full node representations. Additionally, Yoo et al. [38] proposed the structured variational graph autoencoder, which utilizes structured variational inference to regulate the distribution of latent variables. Xia et al. [35] introduced the attribute imputation autoencoders for attribute-missing graphs.

Existing attribute completion methods focus solely on completion accuracy or downstream task performance, neglecting the critical issue of class imbalance. In contrast, our work tackles both class imbalance and missing attributes.

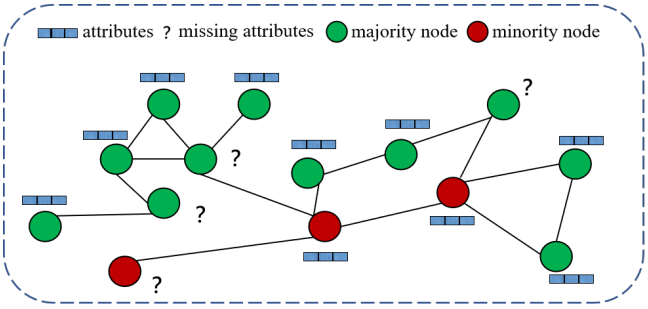


Figure 3: An example of a class-imbalanced graph with missing attributes.

3 Problem Definition

Definition 1 (Class-imbalanced graph with missing attributes). *$\mathbf{G} = (\boldsymbol{V}, \boldsymbol{X}^o, \boldsymbol{A}, \boldsymbol{C})$ is a class-imbalanced graph with missing attributes that contains the node set $\boldsymbol{V} = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_N\}$, observed node attribute matrix $\boldsymbol{X}^o \in \mathbb{R}^{N^o \times D}$, class set $\boldsymbol{C} = \{C_1, \ldots, C_k\}$, and adjacent matrix $\boldsymbol{A} \in \{0,1\}^{N \times N}$. Let $\boldsymbol{X}^m \in R^{N^m \times D}$ be a attribute matrix of attribute-missing nodes, where N, N^m, N^o and D represent the number of nodes, attribute-missing nodes, attribute-observed nodes, and node attribute dimensions, respectively. $\boldsymbol{V}^o$ and $\boldsymbol{V}^m$ are the sets of attribute-observed and attribute-missing nodes, respectively. $|C_k|$ is the number of nodes in class k. $\rho = \frac{min_k|C_k|}{max_k|C_k|}$ is imbalance ratio. Accordingly, $N = N^o + N^m$, $\boldsymbol{V} = \boldsymbol{V}^o \cup \boldsymbol{V}^m$, and $\boldsymbol{V}^o \cap \boldsymbol{V}^m = \emptyset$. $\mathbf{G} = (\boldsymbol{V}, \boldsymbol{X}^o, \boldsymbol{A}, \boldsymbol{C})$ is a class-imbalanced graph with missing attributes if ρ is small and N^o is less than the number of graph nodes, i.e., there exists $|C_i| \ll |C_j|$ and $N^o < N$.*

Figure 3 shows an example of a class-imbalanced graph with missing attributes. Based on the notations provided above, the problem of class-imbalanced node classification on attribute-missing graphs is formally defined as follows:

Problem 1. *Given a class-imbalanced graph with missing attributes $\mathbf{G} = (\boldsymbol{V}, \boldsymbol{X}^o, \boldsymbol{A}, \boldsymbol{C})$, we aim to build a model f over $\mathbf{G}$ to predict the labels $\boldsymbol{Y}_u$ of unlabeled nodes.*

$$f(\mathbf{G}, \boldsymbol{Y}_t) \rightarrow \boldsymbol{Y}, \tag{1}$$

where $\boldsymbol{Y}_t$ is the label set of the labeled nodes, $\boldsymbol{Y}$ is the label set of all nodes, $\boldsymbol{Y}_t \cup \boldsymbol{Y}_u = \boldsymbol{Y}$.

4 Methodology

4.1 Overall Framework

In this section, we propose a novel model, GraphIAM, for **Problem 1**. GraphIAM is a two-stage model that utilizes a disentangled approach to independently learn node representations during the pre-training phase and to train a classifier during the fine-tuning phase, with the objective of completing missing attributes and mitigating the class imbalance issue. It consists of three components: data-augmentation, node representation learning, and node classification (as show in Figure 2). (1) The data-augmentation module

aims to capture information in the attribute-missing graph while enhancing the robustness of node representations. (2) The node representation learning module aims to learn node representations through graph contrastive learning with oversampling. This module effectively imputes missing attributes and generates minority nodes through oversampling. (3) The node classification module incorporates an adapter to optimize specific tasks, thereby enhancing the generalization ability of the model.

4.2 Data Augmentation

Previous methods for class-imbalanced node classification on graphs often relied on the assumption of complete attribute availability. However, this strong assumption lacks universality, as real-world graphs are typically complex and diverse, often accompanied by missing attributes. Consequently, when dealing with class-imbalanced graphs with missing attributes, more adaptive approaches are required to accurately capture both the structural and attribute information, enhancing the accuracy of node classification.

Graph contrastive learning [37, 39] (GCL) is an unsupervised learning method aimed at learning better and more robust node representations, which aligns with the objective of the pre-train phase in our proposed two-stage model. Therefore, we employ GCL for node representation learning, which generates augmented graph views through data transformations and optimizes representations via contrastive loss. [21, 30, 37]. For GraphIAM, the first step involves constructing augmentation graphs. To facilitate node representation learning and downstream tasks, the augmentation method is applied to transform the original graph into two types of augmented graphs: structural and attribute augmentation graphs.

To address the attribute-missing issue, we introduce a structural augmentation graph. This graph maintains the original graph structure while replacing node attributes using structural information (identity matrix), as illustrated by G-aug1 in Figure 2. This augmentation graph enables GraphIAM to utilize the structural information to complete the missing attributes, thus avoiding noise caused by the introduction of external information.

To obtain robust representations, we introduce an attribute augmentation graph (G-aug2 in Figure 2). It consists of attribute-observed nodes and their edges. Furthermore, the masking technique is employed to randomly mask attributes of some nodes. Specifically, let $\mathbf{V}_m$ be the subset of attribute augmentation graph randomly selected for masking. For each node in $\mathbf{V}_m$, its attributes are replaced with a special mask token, denoted as $\boldsymbol{x}_{[mask]} \in \mathbb{R}^D$. Thus, the masked attribute matrix $\mathbf{X}^{o'}$ can be defined as:

$$\boldsymbol{x}_i^{'} = \begin{cases} \boldsymbol{x}_{[mask]}, & if\ v_i \in \mathbf{V}_m \\ \boldsymbol{x}_i, & if\ v_i \notin \mathbf{V}_m \end{cases}, \tag{2}$$

where $\boldsymbol{x}_i^{'}$ is the augmentation attributes of node v_i. The augmented attribute matrix $\mathbf{X}^{o'}$ is constructed by $\boldsymbol{x}_i^{'}$. The masking technique strengthens graph contrastive learning by enhancing the robustness of node representations, especially for minority nodes, thereby alleviating class imbalance.

The data augmentation strategy not only guarantees the efficacy of the model in addressing class imbalance, but also facilitates the reconstruction of missing attributes, thereby enhancing the learning of superior node representations.

4.3 Node Representation Learning

The aim of this step is to transform the augmented graphs into representations that are complete, class-balanced, and robust. The implementation process is as follows: (1) Encoding, extracting class-balanced and complete representations from the original graph. (2) Decoding, where the attributes of two augmented graphs are independently reconstructed by decoders that share parameters. (3) Loss function, incorporating contrastive loss and link prediction loss to learn node representations.

4.3.1 Encoding stage. To obtain complete and robust representations, the encoder adopts a Graph Convolutional Network (GCN) as the backbone, which is widely used in class-imbalanced and attribute-missing graphs. Given a graph $\mathbf{G}$, the formulation of the GCN encoder is as follows:

$$\mathbf{GCN}(\mathbf{X}, \mathbf{A}) = \sigma(\mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}\mathbf{X}\mathbf{W}), \tag{3}$$

where $\mathbf{A}$, $\mathbf{X}$, $\mathbf{W}$ and $\mathbf{D}$ represent the adjacency matrix, attribute matrix, weight matrix and degree matrix, respectively. $\sigma(.)$ denotes the ReLU activation function.

Specifically, for the structural augmentation graph, a 1-layer GCN is used to learn node representations:

$$\mathbf{H} = \mathbf{GCN}_1(\mathbf{I}, \mathbf{A}), \tag{4}$$

where $\mathbf{I}$ is the identity matrix with structural information. It can obtain the information of the attribute-missing nodes.

To obtain representations of the attribute augmentation graph with masking attributes $\mathbf{X}^{o'}$ and adjacency matrix $\mathbf{A}^o$, we first employ a 1-layer GCN to learn representations:

$$\mathbf{Z}^o = \mathbf{GCN}_2(\mathbf{X}^{o'}, \mathbf{A}^o). \tag{5}$$

To obtain the complete attributed graph, we select the attribute-missing representations of the structural augmentation graph $\mathbf{H}^m \in \mathbf{H}$, and then recompose them with $\mathbf{Z}^o$:

$$\mathbf{Z}_1 = \mathbf{Concat}(\mathbf{H}^m, \mathbf{Z}^o). \tag{6}$$

Then $\mathbf{Z}_1$ is fed into another 1-layer GCN to obtain the final node representation $\mathbf{Z}_2$:

$$\mathbf{Z}_2 = \mathbf{GCN}_3(\mathbf{Z}_1, \mathbf{A}). \tag{7}$$

To address class imbalance and achieve relatively balanced classes, we use the oversampling technique SMOTE [3] to obtain synthetic minority nodes from both the structural and attribute augmentation graphs, and connect these synthetic nodes to the graph for class balance. The functions are as follows:

$$\boldsymbol{h}_{v'} = (1-\gamma)\cdot\boldsymbol{h}_v + \gamma\cdot\boldsymbol{h}_{nn(v)}, \tag{8}$$

and

$$nn(v) = \underset{u\in\{\mathbf{V}/v\}, y_u = y_v}{argmin} \|\boldsymbol{h}_u - \boldsymbol{h}_v\|, \tag{9}$$

where v' is the synthetic minority node. We conduct imputation on a given node v, by utilizing its nearest neighbor $nn(v)$. $nn(v)$ is identified using the Euclidean distance metric among same-class neighbors. $\boldsymbol{h}_v$ is the representation of node v. γ is a uniformly distributed random variable within the interval from 0 to 1. y_u and y_v are the labels of nodes u and v, respectively. The node v' shares the same label as v. We control the number of synthetic nodes per minority class with an oversampling scale hyperparameter.

After applying the oversampling, we obtain the final node representations for both the structural and attribute augmentation graphs, denoted as $\mathbf{H}^{'}$ and $\mathbf{Z}^{'}$, respectively.

4.3.2 Decoding stage. The goal of the decoding stage is to map the representations back to the attribute and adjacency matrix, ensuring better representation learning across all attribute-missing and attribute-observed nodes. Before decoding, we re-mask the masked nodes in the attribute augmentation graph. Specifically, for each node in $\mathbf{V}_m$, we replace its node representations from $\mathbf{Z}_2$ with mask token $\boldsymbol{z}_{[mask]}$. The re-masking mechanism requires masked nodes to integrate neighborhood information, thereby enabling more accurate reconstruction of their original attributes.

We feed the representations $\mathbf{H}^{'}$ and $\mathbf{Z}^{'}$ into the parameter-shared multi-layer perceptron (MLP) decoder, obtaining the reconstructed attributes $\mathbf{X}_H$ and $\mathbf{X}_Z$. The function is:

$$\mathbf{X}_H = \mathbf{MLP}(\mathbf{H}^{'}), \mathbf{X}_Z = \mathbf{MLP}(\mathbf{Z}^{'}). \tag{10}$$

4.3.3 Loss function. In the pre-training stage of GraphIAM, the loss function guides the model to learn better node representations by minimizing the difference between the original and reconstructed graph, as well as minimizing the contrast between the two reconstructed graphs derived from the same graph. We define the loss function into two parts: contrastive loss and link prediction loss.

We utilize the cross-entropy loss for the contrastive loss function. The formulation is:

$$\mathcal{L}_{CL} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{D} x_{ij} log(x_{h_{ij}}) - \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{D} x_{ij} log(x_{z_{ij}}), \tag{11}$$

where x_{ij}, $x_{h_{ij}}$, and $x_{z_{ij}}$ are the elements of $\mathbf{X}$, $\mathbf{X}_H$, and $\mathbf{X}_Z$, respectively. Through contrastive loss, robust representations are generated, while the model is encouraged to minimize the distance between the distribution of reconstructed attributes and the real attribute distribution. Additionally, we reconstruct the graph using inner products to obtain more structural information:

$$p(\mathbf{A}'|\mathbf{Z}^{'}) = \prod_{i=1}^{N}\prod_{j=1}^{N} p(a'_{ij}|\boldsymbol{z}^{'}_i, \boldsymbol{z}^{'}_j), \tag{12}$$

$$p(a'_{ij} = 1|\boldsymbol{z}^{'}_i, \boldsymbol{z}^{'}_j) = \delta(\boldsymbol{z}^{'\top}_i \boldsymbol{z}^{'}_j), \tag{13}$$

where a'_{ij} is the element of $\mathbf{A}'$, and $\delta(\cdot)$ represents the sigmoid function. The link prediction loss function is:

$$\mathcal{L}_{\mathrm{A}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{N}(\boldsymbol{a}_{\mathrm{ij}} - \boldsymbol{a'}_{\mathrm{ij}})^2. \tag{14}$$

Once the model is trained, the representation $\mathbf{Z}^{'}$ obtained from the encoder can be used for node classification tasks.

4.4 Node Classification

To enhance the model's generalization ability and achieve high accuracy in node classification, we adjust the parameters through fine-tuning to adapt to node classification tasks. This stage involves fine-tuning the adapter and learning the parameters for the node classification task.

4.4.1 Adapter. We propose using the adapter [8, 15] to fine-tune parameters, enabling learned node representations to adapt to downstream node classification tasks. As shown in Figure 2, we design trainable adapters to integrate task-specific knowledge with general knowledge from the pre-trained model.

Specifically, the adapter utilizes a bottleneck architecture, consisting of a down-projection function $\mathbf{H}_d = \mathbf{GCN}_D(\mathbf{I}, \mathbf{A}), \mathbf{H}_d \in \mathbb{R}^{n_d}$, a ReLU activation function, and an up-projection function $\mathbf{H}_u = \mathbf{MLP}_D(\mathbf{H}_d), \mathbf{H}_u \in \mathbb{R}^{n_u}$. To reduce the parameter space, the intermediate dimension n_d is kept small as a bottleneck. The adapter function is as follows:

$$\mathbf{H}_u = \Phi(\mathbf{I}, \mathbf{A}) = \mathbf{MLP}_D(\mathbf{ReLU}(\mathbf{GCN}_D(\mathbf{I}, \mathbf{A}))). \tag{15}$$

The representations after fine-tuning with the adapter are obtained by combining the output of the adapter with the original representations through element-wise addition.

$$\mathbf{Z}_3 = (1-\alpha)\mathbf{Z}_2 + \alpha\mathbf{H}_u. \tag{16}$$

The initialization parameters of $\Phi(.)$ are obtained through the pre-training stage. The hyperparameter α ranges from $[0, 1]$, aiming to prevent catastrophic forgetting.

To our knowledge, we are the first to use adapters to enhance the generalization of a two-stage model on class-imbalanced graphs, which addresses the mismatch between node representation learning and downstream tasks.

4.4.2 Classifier. To utilize structural information, we use a GCN and a linear layer as the classifier. The classifier based on the GCN can be expressed as follows:

$$p_c = Softmax(\mathbf{GCN}(\boldsymbol{z}_{3_v}, (\boldsymbol{z}_{3_{v'}} : v^{'} \in N(v)))), \tag{17}$$

where $N(v)$ denotes the neighbors of node v, p_c denotes the probability distribution of class labels for a given node v. The optimization of the classifier is achieved through the utilization of the cross-entropy loss function:

$$\mathcal{L}_{\mathrm{C}} = \sum_{v\in\hat{\mathbf{V}}}\sum_{c}(\mathbf{1}(\hat{Y}_v == c) \cdot log p_c[c]), \tag{18}$$

where $\hat{\mathbf{V}}$ denotes the set of labeled nodes, $\hat{Y}_v$ is a one-hot vector representing the ground-truth labels.

4.5 Training and Optimization

We adopt a two-stage training approach to train the proposed model. In the first stage, we learn class-balanced and attribute-completed node representations by minimizing the loss function from the pre-training stage. This includes contrastive loss and link prediction loss. The loss function of pre-training stage is:

$$\mathcal{L}_{pre-train} = \mathcal{L}_{CL} + \beta\mathcal{L}_A. \tag{19}$$

By minimizing this loss, our model not only completes missing attributes but also generates the synthetic representations of minority nodes. Ultimately, it obtains a complete and class-balanced representation, which can effectively be used for class-imbalance graphs with missing attributes.

In the second stage, we predict the classes through the adapter and node classification loss. The loss function is

$$\mathcal{L}_{down-task} = \mathcal{L}_C. \tag{20}$$

Table 1: A summary of graph datasets.

Datasets	Classes	Nodes	Edges	Features	Imbalance ratio
Cora	7	2708	5429	1433	0.22
Citeseer	6	3327	4732	3703	0.36
Pubmed	3	19717	44338	500	0.52
Amap	7	7650	119081	745	0.36
Amac	9	13752	245861	767	0.06

5 Experimental Analysis

5.1 Experimental Setup

5.1.1 Datasets. Drawing upon previous research, we evaluate our model GraphIAM on several widely used citation datasets [24] (Cora, Citeseer, Pubmed) and co-purchasing datasets (Amap, Amac) [1] for node classification tasks. Dataset details are shown in Table 1. We generate attribute-missing graphs by randomly selecting 60% of the nodes and eliminating their attributes.

- For the citation dataset, we simulate class imbalance following the step imbalance setting [2]. Specifically, for Cora and Citeseer, three classes are randomly chosen as minority classes, while for Pubmed, one class is designated as the minority class. In the training set, the majority classes retain 20 labeled nodes, whereas the number of nodes for minority classes is $20 \times \rho_c$, with ρ_c denoting the class imbalance ratio (default value of ρ_c is 0.5). During validation and testing, an equal number of nodes are sampled for all classes to ensure balance between the validation and test sets.
- (2) For the co-purchase dataset, since it is inherently class imbalanced, we employ a naturally imbalanced method to partition the data into training, validation, and test sets. Specifically, 25% of the nodes in each class are used for training, 25% for validation, and the remaining 50% for testing.

5.1.2 Baselines. We select two categories of class-imbalanced baselines, particularly those that have been recognized as the state-of-the-art. (1) For representative class-imbalanced approaches, we compare (a) Vanilla [14], which directly uses GCN; (b) Re-weight [40], which assigns higher weights to minority nodes; (c) SMOTE [3] and Embed-SMOTE [1], which generate synthetic minority nodes through imputation techniques; (d) Oversampling, which balances the data by replicating the minority nodes. (2) For graph methods including two-stage methods, we compare (a) two-stage methods, GraphSMOTE [42] and GraphMixup [33], GraphDAO [36], which introduce SMOTE and Mixup to generate minority nodes, and (b) GraphSR [43], which utilizes a pseudo-labeling technique to augment minority nodes.

5.1.3 Evaluation metrics. In accordance with previous research on evaluating class-imbalanced graph methods [33, 42, 43], we utilize three metrics, namely classification accuracy (ACC), mean area under the receiver operating characteristic curve (AUC-ROC), and F Score, to assess the efficacy of our proposed model. While ACC is a widely used metric that ensures effective classification of the majority classes, the model's effectiveness on the minority classes is inadequately reflected. Therefore, to better reflect the performance on the minority classes, we incorporate the F Score and AUC-ROC metrics. Specifically, AUC-ROC measures the area under the ROC curve, while the F Score provides the harmonic mean of precision and recall for every class.

5.1.4 Implementation details. The experiments are conducted on a server equipped with a NVIDIA GeForce GTX GPU. We implement the proposed GraphIAM using PyTorch. Except for GraphSR [43] and GraphDAO [36], which employs the same experimental settings as described in the original paper, the settings of other baselines are obtained from the GraphMixup [33] open-source library[2]. For our model, the hyperparameters are determined via grid search, with a dropout rate of 0.2 and a learning rate of 0.001. Furthermore, the weight decay is configured to 5e-4. The hyperparameters β and α are set to 0.9 and 0.2, respectively. The latent dimension is 128, the bottleneck dimension $n_d \in [1, 128]$. The training procedure consists of no fewer than 1000 epochs, with the Adam optimization algorithm employed to optimize the model. We repeat the experiment five times and average the results.

5.2 Overall Performance

In this section, we evaluate GraphIAM and baselines on citation datasets and co-purchasing datasets for class-imbalanced node classification with missing attributes. For citation datasets, we set the oversampling scale to 1.0, the imbalanced ratio to 0.5, and the attribute missing ratio to 0.6. "-" indicates out of memory. The results are shown in Table 2.

Results & discussion. From Table 2, we have several observations: (1) **In scenarios involving attribute missingness, methods designed for class-imbalanced node classification on graphs often fail to achieve optimal performance**. For example, for the Cora and Pubmed, representative class-imbalance methods consistently outperform those designed for class-imbalanced graphs. This indicates that existing methods for class-imbalanced graphs are insufficient in dealing with the attribute-missing issue. (2) **GraphIAM outperformed other baselines on almost all datasets**. This is because our proposed two-stage model completes the missing attributes during the pre-training phase. Furthermore, we incorporate adapters to enhance the model's generalization for downstream node classification tasks.

For co-purchasing datasets Amap and Amac. We set the oversampling scale to 1.0, the class imbalance ratio to 0.5, and the attribute missing ratio to 0.6. The results are shown in Table 3 and Table 4.

Results & discussion. From Table 3 and Table 4, we have several observations: (1) **Consistent with the experimental results observed on the citation datasets, GraphIAM on the co-purchasing datasets reveal that methods tailored for class-imbalanced graphs often fail to attain optimal performance, and even underperform classic class-imbalanced methods in the attribute-missing graphs.** These results reveal that existing class-balancing methods exhibit degraded efficacy when applied to naturally class-imbalanced graphs with missing attributes. (2) **In the naturally class-imbalanced datasets, GraphIAM outperformed other baselines in almost all datasets.** This validates

[1] https://docs.dgl.ai/api/python/dgl.data.html#amazon-co-purchase-dataset

[2] https://github.com/LirongWu/GraphMixup

Table 2: ACC, AUC-ROC and F Score values for class-imbalanced node classification on citation graphs.

Methods	Cora			Citeseer			Pubmed		
	ACC	AUC-ROC	F Score	ACC	AUC-ROC	F Score	ACC	AUC-ROC	F Score
Vanilla	0.6857	0.9004	0.6848	0.4051	0.7376	0.4098	0.7535	0.8837	0.7525
Oversampling	0.6692	0.8973	0.6709	0.4151	0.7418	0.4189	0.7696	0.8934	0.7710
Re-weight	0.6803	0.8999	0.6827	0.4303	0.7419	0.4385	0.7757	0.8950	0.7751
SMOTE	0.6797	0.8998	0.6809	0.4161	0.7424	0.4247	0.7656	0.8885	0.7671
Embed-SMOTE	0.6865	0.8935	0.6872	0.4272	0.7324	0.4417	0.7616	0.8813	0.7622
GraphSMOTE	0.6753	0.8937	0.6767	0.3616	0.6875	0.3741	0.7575	0.8886	0.7583
GraphMixup	0.6562	0.8779	0.6574	0.4272	0.7421	0.4266	-	-	-
GraphSR	0.6816	0.8809	0.6836	0.4721	0.7595	0.4472	0.6594	0.8327	0.6577
GraphDAO	0.6856	0.8977	0.6852	0.4303	0.7537	0.4456	-	-	-
Ours	**0.6952**	**0.9018**	**0.6920**	**0.4879**	**0.7788**	**0.4889**	**0.7798**	**0.8973**	**0.7797**

Table 3: ACC, AUC-ROC and F Score for class-imbalanced node classification on Amap dataset.

Methods	Amap		
	ACC	AUC-ROC	F Score
Vanilla	0.8658	0.9772	0.8659
Oversampling	0.8559	0.9766	0.8552
Re-weight	0.8616	0.9742	0.8617
SMOTE	0.8631	0.9742	0.8618
Embed-SMOTE	0.8573	0.9771	0.8576
GraphSMOTE	0.8322	0.9712	0.8265
GraphMixup	0.8397	0.9745	0.8375
GraphDAO	0.8236	0.9688	0.8210
Ours	**0.8727**	**0.9781**	**0.8723**

Table 4: ACC, AUC-ROC and F Score for class-imbalanced node classification on Amac dataset.

Methods	Amac		
	ACC	AUC-ROC	F Score
Vanilla	0.8389	0.9712	0.8092
Oversampling	0.8349	0.9689	0.8039
Re-weight	0.8372	0.9697	0.8063
SMOTE	0.8337	0.9681	0.8048
Embed-SMOTE	0.8266	0.9685	0.7861
GraphSMOTE	0.8047	0.9701	0.7552
GraphMixup	0.8240	0.9750	0.7840
GraphDAO	0.8065	0.9699	0.7311
Ours	**0.8423**	**0.9724**	**0.8166**

the effectiveness and robustness of our proposed method on class-imbalanced and attribute-missing graphs.

5.3 Impact of Different Class Imbalance Ratios

To evaluate the robustness of GraphIAM in relation to the variation of class imbalance ratios, we maintain a fixed oversampling scale of 1.0 and an attribute missing ratio of 0.6, while varying the class imbalance ratio across values of 0.1, 0.3, 0.5, and 0.7. We conduct comparisons of the F Scores between GraphIAM and baselines on Cora and Citeseer datasets. The experimental results on the node classification task under different class imbalance ratios are presented in Table 5.

Results & discussion. Based on the results, we draw the following observations. (1) **The performance of most algorithms improves with an increase in the class imbalance ratio** ρ_c. The number of minority nodes in training set is $20 \times \rho_c$. As ρ_c increases, the graph becomes more balanced, thereby enhancing the overall effectiveness of the algorithms in the node classification task. (2) **Our model is robust than other baselines.** As the class imbalance ratio increases, the performance of node classification for GraphIAM consistently outperforms the other baselines. These results verify the robustness of our model GraphIAM in relation to varying class imbalance ratios. (3) **The improvement achieved by GraphIAM is more substantial when the imbalance is more extreme**. For example, in the Citeseer dataset, when the class imbalance ratio is 0.1, GraphIAM outperforms the second-best baseline by 3.2%, whereas when the class imbalance ratio reaches 0.7, the margin reduces to 0.028%. This observation demonstrates the superior effectiveness of our model GraphIAM in scenarios of severe class imbalance.

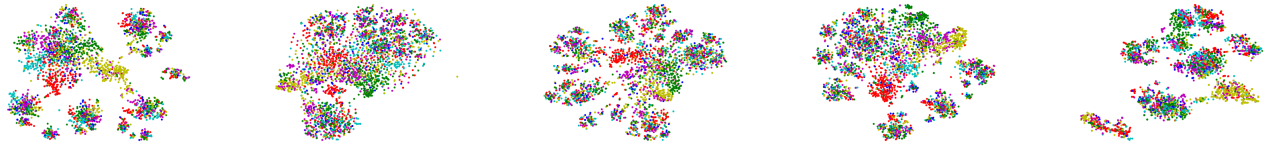
5.4 Case Study

We provide a case study that compares the classification accuracy of GraphIMA with the baselines on different attribute missing ratios in the Cora and Citeseer. We set the oversampling scale to 1.0, the class imbalance ratio to 0.5. The results are shown in Figure 5.

From the results, we have several observations: (1) **The performance of most algorithms reduces with an increase of the attribute missing ratio**. As the attribute missing ratio increases, the graph misses more attributes, thereby reducing the node classification accuracy. (2) **Our model exhibits greater robustness across varying ratios of the missing attributes**. As the missing attribute ratio increases, baselines suffer a sharp performance decline, whereas our method remains stable in node classification. (3) **The improvement achieved by GraphIAM is more substantial when the missing attribute is more extreme**. With low missing ratios, some baselines surpass GraphIAM, but GraphIAM consistently outperforms others at higher ratios. For example, in Cora,

Table 5: Performances under different class imbalance ratios.

Methods	Class Imbalance Ratio on Cora				Class Imbalance Ratio on Citeseer			
	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7
Vanilla	0.4823	0.6056	0.6848	0.7044	0.2910	0.3764	0.4098	0.4672
Oversampling	0.4595	0.5945	0.6709	0.7039	0.2830	0.3685	0.4189	0.4692
Re-weight	0.4781	0.6112	0.6827	0.7047	0.2886	0.3762	0.4385	0.4679
SMOTE	0.4285	0.5990	0.6809	0.7031	0.2744	0.3693	0.4247	0.4665
Embed-SMOTE	0.4896	0.6303	0.6872	0.6940	0.3044	0.3921	0.4417	0.4507
GraphSMOTE	0.5533	0.6541	0.6767	0.6699	0.3004	0.3819	0.3741	0.4098
GraphMixup	0.5491	0.6352	0.6574	0.6840	0.3223	0.3284	0.4266	0.4404
GraphSR	0.4627	0.6575	0.6836	0.6851	0.3162	0.4302	0.4472	0.4895
GraphDAO	0.5587	0.6392	0.6852	0.6882	0.3345	0.3899	0.4456	0.4419
Ours	**0.5725**	**0.6612**	**0.6920**	**0.7058**	**0.3662**	**0.4727**	**0.4889**	**0.4923**



(a) Clean version **(b) Vanilla** **(c) Oversampling** **(d) GraphDAO** **(e) GraphIAM**

Figure 4: Visualization of the Cora dataset.

when the attribute missing ratio is 0.4, the F Score of GraphIAM is lower than GraphSR. However, as the attribute missing ratio increases, the F Score of GraphIAM shows a notable improvement. These results highlight that our problem is not a simple incremental addition. Morever over, GraphIAM is effective and robust to class-imbalance graphs with missing attributes.

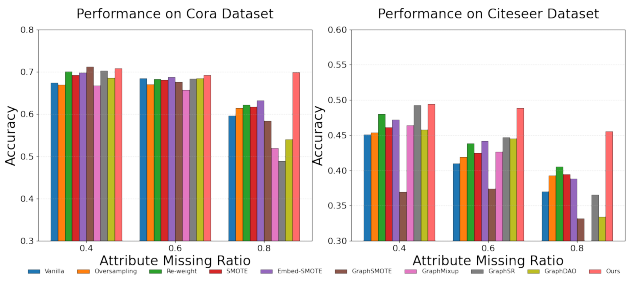


Figure 5: Classification accuracy on different attribute missing ratios.

5.5 Graph Visualization

We employ 2D t-SNE [27] to compare the node representation visualization performance of the GraphIAM and baselines on the Cora. The experimental results are presented in Figure 4.

We observed that: (1) Compared to Figure 4 (a) (Vanilla under complete attributes and class-balanced conditions), both the Vanilla (Figure 4 (b)) and class imbalance mitigation methods (Figure 4 (c) and Figure 4 (d)) demonstrate significantly degraded visualization performance when handling class imbalance with missing attributes. This confirms that both class imbalance and attribute absence adversely affect node representation learning, while conventional class-balancing methods show limited efficacy in attribute-missing scenarios. (2) Compared to other methods, our proposed model GraphIAM (Figure 4 (e)) accounts for the missing attributes, leading to better results.

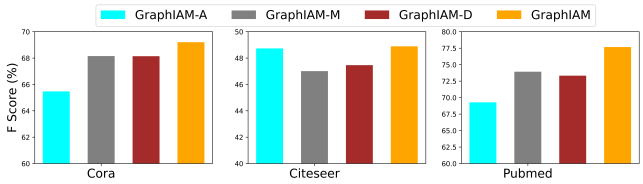


Figure 6: Component ablation.

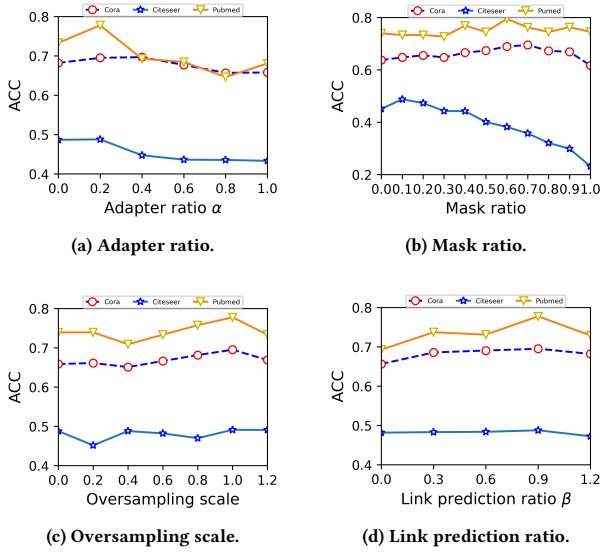
5.6 Ablation Study

To evaluate the contribution of each component in GraphIAM, we conduct an ablation study by removing individual modules and comparing our model GraphIAM with its variants: GraphIAM-A (no link prediction loss), GraphIAM-M (no masking mechanism), and GraphIAM-D (no adapter).

Figure 6 illustrates the ablation experiments on the Cora, Citeseer, and Pubmed. The results indicate that the accuracy of GraphIAM is compromised when any of the key components are removed, highlighting the importance of each component.

5.7 Hyperparameter Analysis

Figure 7 reports the hyperparameter analysis for Cora, Citeseer and Pubmed. Specifically, we evaluated the node classification performance of GraphIMA by varying the hyperparameter of adapter ratio α, mask ratio, oversampling scale, and the hyperparameter of link prediction β in the ranges of 0.0 to 1, 0.0 to 1, 0.0 to 1.2, and 0.0 to 1.2, respectively.



(a) Adapter ratio. (b) Mask ratio.

(c) Oversampling scale. (d) Link prediction ratio.

Figure 7: Hyperparameters analysis of GraphIMA.

From Figure 7, it is evident that GraphIMA achieves optimal performance across nearly all datasets when $\alpha = 0.2$, oversampling scale = 1.0, $\beta = 0.9$. The performance of GraphIMA exhibits a pattern of initially increasing and subsequently decreasing, reaching its peak at a specific value. Particularly, Figure 7(a) illustrates the impact of the adapter on GraphIMA. A high adapter ratio will cause the model to pay more attention to the downstream task, leading to catastrophic forgetting in the model. Figure 7(b) illustrates the impact of the mask ratio on the node classification task for the GraphIMA. We can observe that the optimal mask ratios vary for different datasets, with the optimal mask ratios for Cora, Citeseer, and Pubmed being 0.7, 0.1 and 0.6, respectively. Figure 7 (c) demonstrates the impact of the oversampling scale on GraphIMA. The performance of most algorithms improves with an increase in the oversampling scale. This is because as the oversampling scale increases, the number of synthetic minority nodes increases, making the nodes in the graphs more balanced and thereby enhancing the overall effectiveness of the algorithms in node classification tasks. However, performance decreases when the oversampling scale exceeds 1.0, as an excessive number of synthetic minority nodes can lead to information redundancy. Figure 7 (d) illustrates the impact of structural reconstruction on the node classification task of GraphIMA. The results indicate that structural reconstruction can enhance the model's performance. This is attributed to the fact that structural reconstruction aids the model in learning more structural information to obtain better node representations.

5.8 Time Efficiency Analysis

We validate the computational complexity through runtime (training & testing). Specifically, we evaluate the overall time consumption of both the GraphIAM and baselines on the Cora, Citeseer and Pubmed. The results are shown in Figure 8.

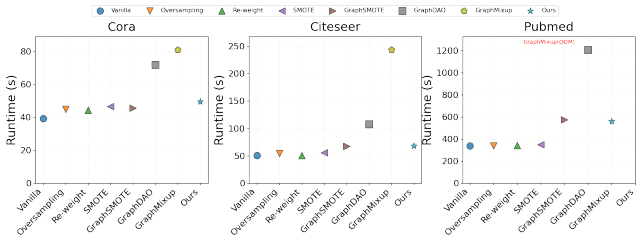


Figure 8: Runtime comparison on different datasets. GraphMixup(OOM) denotes that the GraphMixup exceeded available memory resources.

We observe that the runtime of our model GraphIAM is lower than that of most two-stage models (GraphSMOTE, GraphMixup and GraphDAO). Notably, compared to GraphMixup and GraphDAO, our model exhibits significantly lower runtime than these two methods. Furthermore, the runtime of our model is comparable to that of simpler baseline models (such as Oversampling and SMOTE). Thus, GraphIAM's time complexity is acceptable. More importantly, as evidenced in Table 5 and Figure 5, our model achieves significantly better performance under challenging conditions of high class imbalance and attribute missing rates.

6 Conclusion

In this paper, we introduce a novel problem, namely, class-imbalanced node classification on attribute-missing graphs. To address this challenge, we propose GraphIAM, a two-stage algorithm that simultaneously tackles missing attributes and class imbalance. Specifically, we leverage graph contrastive learning and oversampling techniques to complete missing attributes and learn class-balanced representations. Furthermore, an adapter mechanism is incorporated to enhance the generalization ability of GraphIAM. Experimental results on three real-world datasets demonstrate the superior performance of GraphIAM compared to baselines in terms of the class-imbalanced node classification task.

Acknowledgments

This work is supported by the National Natural Science Foundation of China through grants 62202200 and 62402197, the Inner Mongolia University High-level Talent Project through grant 10000-23112101/286, the Inner Mongolia Autonomous Region Natural Science Foundation through grant 2025QN06010, and the fund of Supporting the Reform and Development of Local Universities (Disciplinary Construction) and the special research project of First-class Discipline of Inner Mongolia A. R. of China under Grant YLXKZX-ND-036.

GenAI Usage Disclosure

No GenAI tools were utilized at any stage of this research.

References

[1] Shin Ando and Chun-Yuan Huang. 2017. Deep over-sampling framework for classifying imbalanced data. In *proceedings of ECML PKDD*, Vol. 10534. 770–785.

[2] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Aréchiga, and Tengyu Ma. 2019. Learning imbalanced datasets with label-distribution-aware margin loss. In *Proceedings of NeurIPS*. 1565–1576.

[3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357.

[4] Deli Chen, Yankai Lin, Guangxiang Zhao, Xuancheng Ren, Peng Li, Jie Zhou, and Xu Sun. 2021. Topology-imbalance learning for semi-supervised node classification. In *Proceedings of NeurIPS*. 29885–29897.

[5] Xu Chen, Siheng Chen, Jiangchao Yao, Huangjie Zheng, Ya Zhang, and Ivor W. Tsang. 2022. Learning on attribute-missing graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 2 (2022), 740–757.

[6] Yue Fan, Dengxin Dai, Anna Kukleva, and Bernt Schiele. 2022. CoSSL: Co-learning of representation and classifier for imbalanced semi-supervised learning. In *Proceedings of CVPR*. IEEE, 14554–14564.

[7] Mahsa Ghorbani, Anees Kazi, Mahdieh Soleymani Baghshah, Hamid R. Rabiee, and Nassir Navab. 2022. RA-GCN: graph convolutional network for disease prediction problems with imbalanced data. *Medical Image Anal.* 75 (2022), 102272.

[8] Anchun Gui, Jinqiang Ye, and Han Xiao. 2024. G-Adapter: Towards structure-aware parameter-efficient transfer learning for graph transformer networks. In *Proceedings of AAAI*. 12226–12234.

[9] Dongxiao He, Chundong Liang, Cuiying Huo, Zhiyong Feng, Di Jin, Liang Yang, and Weixiong Zhang. 2024. Analyzing Heterogeneous Networks With Missing Attributes by Unsupervised Contrastive Learning. *IEEE Trans. Neural Networks Learn. Syst.* 35, 4 (2024), 4438–4450.

[10] Zhenhua Huang, Yinhao Tang, and Yunwen Chen. 2022. A graph neural network-based node classification model on class-imbalanced graph data. *Knowledge-Based Systems* 244 (2022), 108538.

[11] Di Jin, Cuiying Huo, Chundong Liang, and Liang Yang. 2021. Heterogeneous graph neural network via attribute completion. In *Proceedings of WWW*. 391–400.

[12] Di Jin, Rui Wang, Tao Wang, Dongxiao He, Weiping Ding, Yuxiao Huang, Longbiao Wang, and Witold Pedrycz. 2022. Amer: A new attribute-missing network embedding approach. *IEEE Transactions on Cybernetics* (2022), 1–14. Issue 2168-2267.

[13] Bingyi Kang, Saining Xie, Marcus Rohrbach, Zhicheng Yan, Albert Gordo, Jiashi Feng, and Yannis Kalantidis. 2020. Decoupling representation and classifier for long-tailed recognition. In *Proceedings of ICLR*.

[14] Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *Proceedings of ICLR*. 1–1.

[15] Shengrui Li, Xueting Han, and Jing Bai. 2024. AdapterGNN: Efficient delta tuning improves generalization ability in graph neural networks. In *Proceedings of AAAI*. 1–9.

[16] Wen-Zhi Li, Chang-Dong Wang, Hui Xiong, and Jian-Huang Lai. 2023. GraphSHA: synthesizing harder samples for class-imbalanced node classification. In *Proceedings of KDD*. 1328–1340.

[17] Xiaohe Li, Lijie Wen, Yawen Deng, Fuli Feng, Xuming Hu, Lei Wang, and Zide Fan. 2022. Graph neural network with curriculum learning for imbalanced node classification. *arXiv preprint arXiv:2202.02529* (2022).

[18] Yu Li, Tao Wang, Bingyi Kang, Sheng Tang, Chunfeng Wang, Jintao Li, and Jiashi Feng. 2020. Overcoming classifier imbalance for long-tail object detection with balanced group softmax. In *Proceedings of CVPR*. 10988–10997.

[19] Zemin Liu, Yuan Li, Nan Chen, Qian Wang, Bryan Hooi, and Bingsheng He. 2023. A survey of imbalanced learning on graphs: problems, techniques, and future directions. *arXiv preprint arXiv:2308.13821* (2023).

[20] Yihong Ma, Yijun Tian, Nuno Moniz, and Nitesh V. Chawla. 2023. Class-imbalanced learning on graphs: A survey. *arXiv preprint arXiv:2304.04300* (2023).

[21] Costas Mavromatis and G. Karypis. 2021. Graph infoclust: Maximizing coarse-grain mutual information in graphs. In *Proceedings of PAKDD*.

[22] Joonhyung Park, Jaeyun Song, and Eunho Yang. 2022. GraphENS: neighbor-aware ego network synthesis for class-imbalanced node classification. In *Proceedings of ICLR*. 1–1.

[23] Liang Qu, Huaisheng Zhu, Ruiqi Zheng, Yuhui Shi, and Hongzhi Yin. 2021. ImGAGN: imbalanced network embedding via generative adversarial graph networks. In *Proceedings of KDD*. ACM, 1390–1398.

[24] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI Magazine* 29, 3 (2008), 93–106.

[25] Min Shi, Yufei Tang, Xingquan Zhu, David A. Wilson, and Jianxun Liu. 2020. Multi-class imbalanced graph convolutional network learning. In *Proceedings of IJCAI*. 2879–2885.

[26] Shuhao Shi, Kai Qiao, Chen Chen, Jie Yang, Jian Chen, and Bin Yan. 2024. Over-Sampling Strategy in Feature Space for Graphs based Class-imbalanced Bot Detection. In *Companion Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, Singapore, May 13-17, 2024*. ACM, 738–741.

[27] Yue Tan, Yixin Liu, Guodong Long, Jing Jiang, Qinghua Lu, and Chengqi Zhang. 2023. Federated learning on non-iid graphs via structural knowledge sharing. In *IAAI*. AAAI Press, 9953–9961.

[28] Wenxuan Tu, Renxiang Guan, Sihang Zhou, Chuan Ma, Xin Peng, Zhiping Cai, Zhe Liu, Jieren Cheng, and Xinwang Liu. 2024. Attribute-Missing Graph Clustering Network. In *Proceedings of AAAI*. AAAI Press, 15392–15401.

[29] Wenxuan Tu, Sihang Zhou, Xinwang Liu, Yue Liu, Zhiping Cai, En Zhu, Changwang Zhang, and Jieren Cheng. 2022. Initializing then refining: A simple graph attribute imputation network. In *Proceedings of IJCAI*. 3494–3500.

[30] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. 2019. Deep graph infomax. In *Proceedings of ICLR*.

[31] Kefan Wang, Jing An, Mengchu Zhou, Zhe Shi, Xudong Shi, and Qi Kang. 2023. Minority-weighted graph neural network for imbalanced node classification in social networks of internet of people. *IEEE Internet of Things Journal* 10, 1 (2023), 330–340.

[32] Zheng Wang, Xiaojun Ye, Chaokun Wang, Yuexin Wu, Changping Wang, and Kaiwen Liang. 2018. RSDNE: exploring relaxed similarity and dissimilarity from completely-imbalanced labels for network embedding. In *Proceedings of AAAI*. 475–482.

[33] Lirong Wu, Jun Xia, Zhangyang Gao, Haitao Lin, Cheng Tan, and Stan Z. Li. 2022. GraphMixup: improving class-imbalanced node classification by reinforcement mixup and self-supervised context prediction. In *proceedings of ECML PKDD (Lecture Notes in Computer Science, Vol. 13716)*. 519–535.

[34] Riting Xia, Huibo Liu, Anchen Li, Xueyan Liu, Yan Zhang, Chunxu Zhang, and Bo Yang. 2025. Incomplete Graph Learning: A Comprehensive Survey. *CoRR* abs/2502.12412 (2025).

[35] Riting Xia, Chunxu Zhang, Anchen Li, Xueyan Liu, and Bo Yang. 2024. Attribute imputation autoencoders for attribute-missing graphs. *Knowl. Based Syst.* 291 (2024), 111583.

[36] Riting Xia, Chunxu Zhang, Yan Zhang, Xueyan Liu, and Bo Yang. 2024. A novel graph oversampling framework for node classification in class-imbalanced graphs. *Sci. China Inf. Sci.* 67, 6 (2024).

[37] Shu Yin, Peican Zhu, Lianwei Wu, Chao Gao, and Zhen Wang. 2024. GAMC: An unsupervised method for fake news detection using graph autoencoder with masking. In *Proceedings of AAAI*. 347–355.

[38] Jaemin Yoo, Hyunsik Jeon, Jinhong Jung, and U Kang. 2022. Accurate node feature estimation with structured variational graph autoencoder. In *Proceedings of KDD*. 2336–2346.

[39] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. In *Proceedings of NeurIPS*, Vol. 33. 5812–5823.

[40] Bo Yuan and Xiaoli Ma. 2012. Sampling + reweighting: Boosting the performance of AdaBoost on imbalanced datasets. In *Proceedings of IJCNN*. IEEE, 1–6.

[41] Liang Zeng, Lanqing Li, Ziqi Gao, Peilin Zhao, and Jian Li. 2023. ImGCL: revisiting graph contrastive learning on imbalanced node classification. In *Proceedings of AAAI*. 11138–11146.

[42] Tianxiang Zhao, Xiang Zhang, and Suhang Wang. 2021. GraphSMOTE: imbalanced node classification on graphs with graph neural networks. In *Proceedings of WSDM*. 833–841.

[43] Mengting Zhou and Zhiguo Gong. 2023. GraphSR: A data augmentation algorithm for imbalanced node classification. In *Proceedings of AAAI*. 4954–4962.